

When Server-Level Load Imbalance Helps: Power Savings without Capacity Loss

Sherif Abdelrazek, Dhyey Thummar, Ahmed Saeed
Georgia Institute of Technology



Typical Cluster Overprovisioning



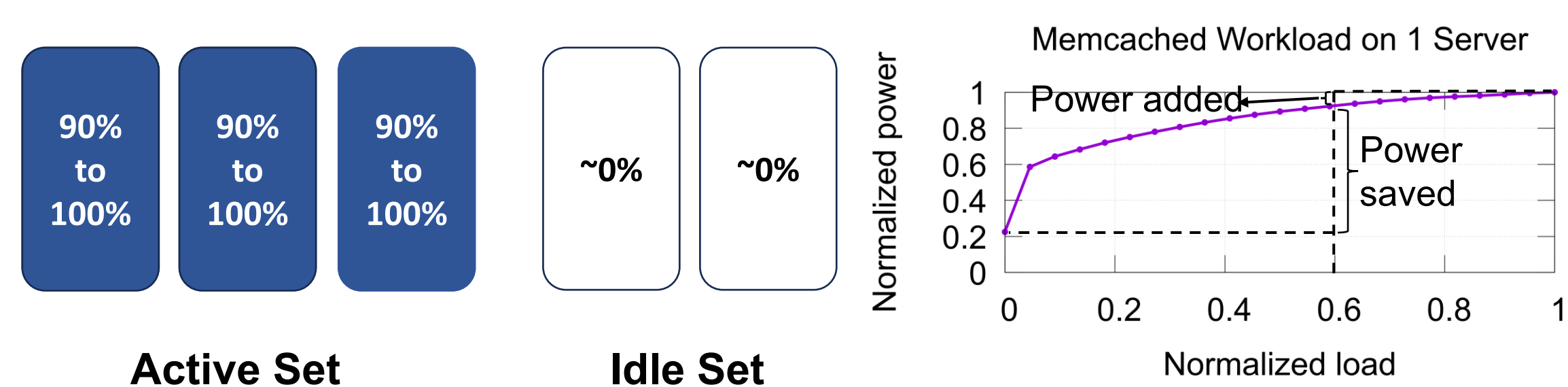
Overprovisioned Servers with Slack

- “Slack” allocated by operators to absorb bursts in demand (**23~48% in Google settings**)
- CPU's lack power proportionality, resulting in **idleness power tax and sublinear power scaling**
- Common load balancing approaches place servers around moderate load

Servers draw a large fraction of peak CPU power even when lightly utilized, so spreading load evenly wastes power

Key Idea: Imbalancing Saves Power

Idea: as CPUs are not power-proportional, shifting work from many lightly-loaded servers to fewer highly utilized servers reduces total power.
In other words, load imbalancing saves power.



Active Set Idle Set

- Leveraging slack, we intentionally imbalance load for cluster-wide power savings by splitting servers into two sets:

Active set: near-capacity servers carry most requests while consuming slightly higher power

Idle set: runs near true idleness consuming considerably lower power

Requirements of Practical Imbalancing

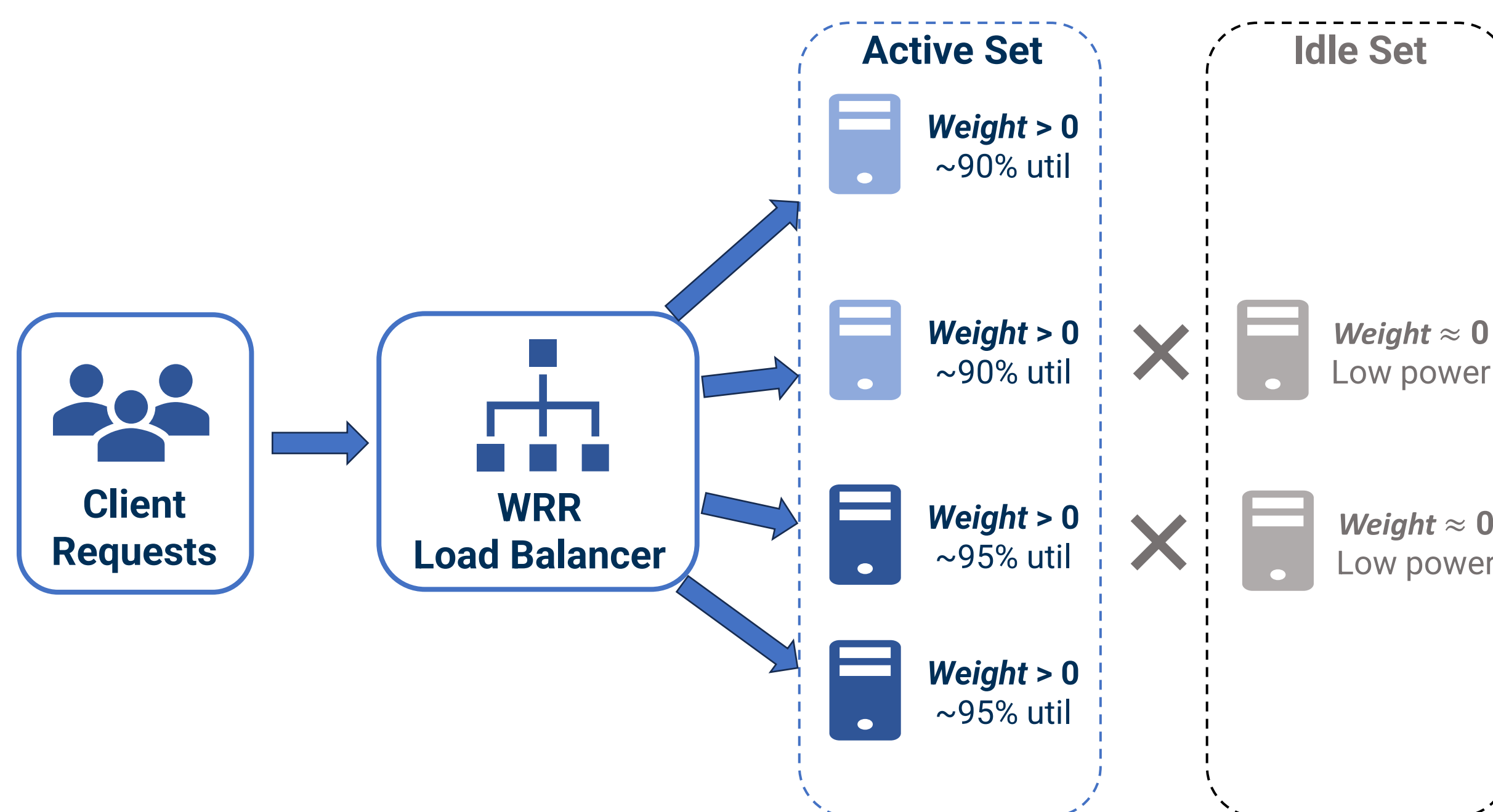
How do we practically realize imbalancing without sacrificing performance?

- Safe hot-set sizing:** keep a high-utilization active set without violating latency SLOs
- Overload detection:** detect rising queueing pressure early enough before tail-latency spikes
- Low-overhead coordination:** select weights/partitions without adding latency

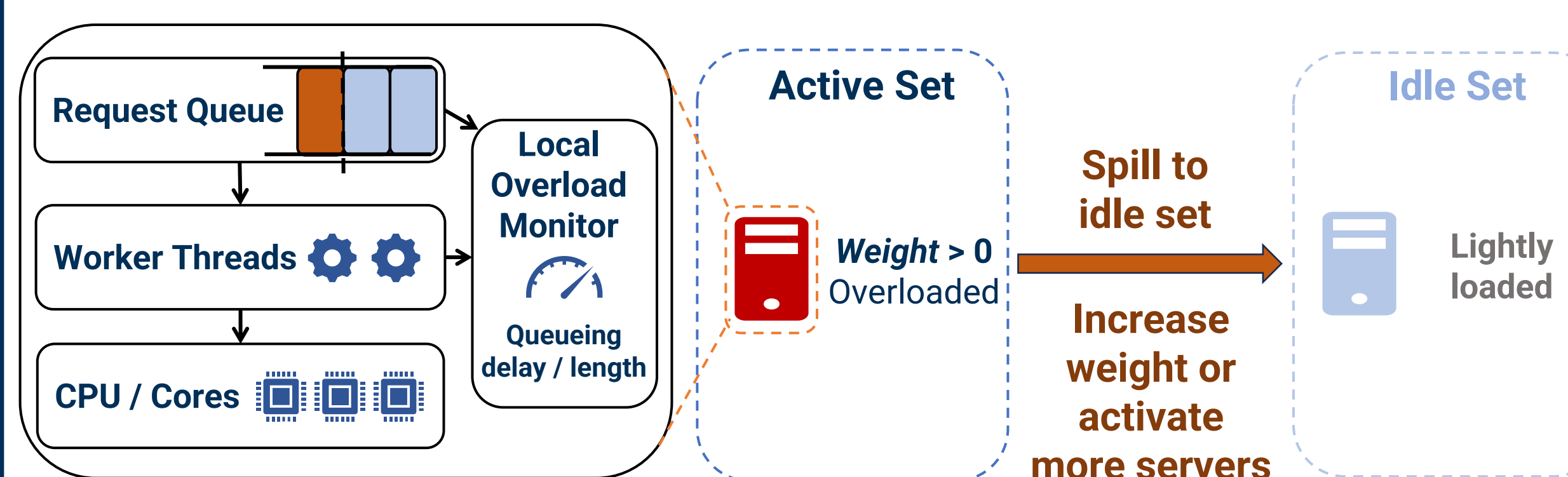
Imbalancing under Static Load

Weighted round-robin (WRR) is a practical solution

- Data plane (biased routing):** weights are configured to keep active servers near full load and the remaining servers idle given a pre-defined aggregate load.
- How weights are set:** assign larger weights to active servers and give very small (or zero) weights to idle servers. Steady-state traffic should concentrate on the active group.



- Control plane (spillover + mode switching):** active servers monitor lightweight overload signals (e.g., queueing delay). When queue delay exceeds a threshold, the system temporarily increases routing to idle servers (or activates more servers) to absorb bursts.



Quantifying the Benefits

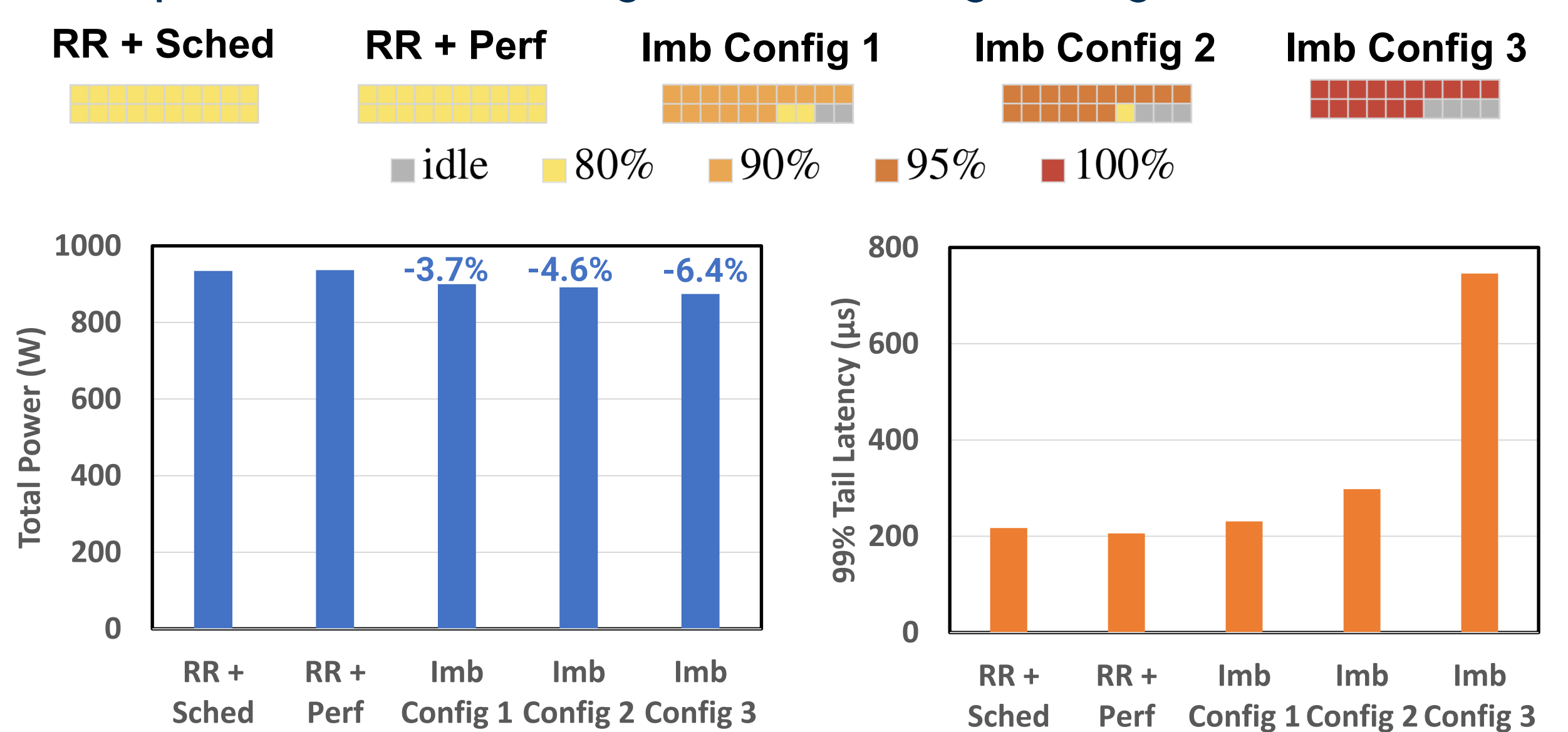
Implementation (Prototype)

- Per-server sidecar process** collects per-server info (e.g., queue delay) and performs overload detection. On overload, it flags the server and notifies the client to trigger spillover.
- Client-side load balancer** routes requests using WRR based on the current configuration. Updates are applied instantly when the server sidecar pushes new weights/modes.

Experimental Setup

- 20 servers, Memcached (ETC), Poisson 80% load
- Baselines: round-robin load balancing + Linux governors (schedutil/performance)

- Experiments emulating 3 imbalancing configurations:



Imbalancing saves power (up to 6.4%) while maintaining tail latency SLOs

Towards an Adaptive Imbalancer

- Track load dynamics:** demand shifts over time; “best” active set and load split must continuously adapt
- Faster overload reaction:** bursts can quickly push active servers into queue buildup; the system must detect overload shed load before latency spikes
- Stability vs agility:** react fast enough to protect SLOs, but avoid oscillations / thrashing from frequent reconfigurations

